AdaptiveFlow Software Engineering Lab

Release 2 — Framework, Knowledgebase & Prototypes

CS212P – Software Systems and Tools Laboratory

Project Overview

AdaptiveFlow is a developer-driven, architecture-first framework for building self-optimizing desktop and web applications. The system observes user behavior using a lightweight, append-only tracker, persists usage statistics in a local database, and periodically evaluates feature utility through a background cycle to enable or disable features via a flat JSON configuration file. No runtime analysis, no code deletion, no parallel heavyweight monitors.

Member	Roll No.
Pranav Tej	CS24B057
Ankit Sinha	CS24B056
Karthik	CS24B046
Bhargav	CS24B028
Nilanjan Singh	CS24B029

Indian Institute of Technology Madras — April 2026

Contents

1 Problem Statement **3**
- 1.1 The Static UI Architecture Problem . . . 3
- 1.2 Why Existing Solutions Are Insufficient . . . 3

2 Proposed Solution **4**
- 2.1 Core Philosophy: Starve, Don't Delete . . . 4
- 2.2 The Four Core Modules . . . 4
  - 2.2.1 Development Guidelines (The Foundation) . . . 4
  - 2.2.2 Statistics and Tracking Module (The Observer) . . . 4
  - 2.2.3 The Flipper Module (The Gatekeeper) . . . 5
  - 2.2.4 The Weekly Adaptation Cycle (The Evaluator) . . . 5
- 2.3 The Feature ID Contract . . . 5
- 2.4 Deliberate Trade-offs . . . 5

3 The Knowledgebase: An Agent-Optimized Architecture **6**
- 3.1 Directory Structure . . . 6
- 3.2 Folder-Level Purposes . . . 7
- 3.3 Agent Traversal Protocol: O(1) Context Scaling . . . 7
- 3.4 The Router as Orchestrator . . . 8
- 3.5 The Memory and Context System . . . 8
- 3.6 Technologies and Stack Coverage . . . 8

4 Prototype Case Studies **9**
- 4.1 Prototype A: NEXUS Baseline (Without Knowledgebase) . . . 9
  - 4.1.1 Application Overview . . . 9
  - 4.1.2 System Architecture . . . 10
  - 4.1.3 Core Feature Modules . . . 10
  - 4.1.4 Heavy Background Work . . . 11
  - 4.1.5 Why This Is the Baseline, Not the Goal . . . 12
- 4.2 Prototype B: NEXUS Adaptive (With Knowledgebase) . . . 12
  - 4.2.1 Why Adaptability Matters for NEXUS . . . 12
  - 4.2.2 Feature Registry . . . 12
  - 4.2.3 Adaptive Metadata Schema . . . 13
  - 4.2.4 Adaptation Layers . . . 14
  - 4.2.5 Adaptive Workflow . . . 16
  - 4.2.6 Flipper Implementation . . . 16
  - 4.2.7 Weekly Evaluator Pseudocode . . . 17
- 4.3 Prototype C: Conduit RealWorld App (Retrofit with Knowledgebase) . . . 17
  - 4.3.1 Application Overview . . . 17
  - 4.3.2 Retrofitting Strategy . . . 18
  - 4.3.3 Feature Registry for Conduit . . . 18
  - 4.3.4 Per-User Schema Model . . . 18
  - 4.3.5 Usage Tracking Pipeline . . . 19
  - 4.3.6 Database Schema for Usage Statistics . . . 19
  - 4.3.7 Adaptation Cycle Trigger . . . 20
  - 4.3.8 Dual-Layer Runtime Enforcement . . . 20

4.3.9 Recovery Interface 20

5 System Architecture Comparison **21**

6 Tech Stack **21**
6.1 Knowledgebase (Stack-Agnostic Documentation) 21
6.2 Prototype B: NEXUS Adaptive (Primary Desktop App) 22
6.3 Prototype C: Conduit Retrofit (Full-Stack Web App) 23

7 Team Roles **23**

8 AI Usage Declaration **24**
8.1 Ankit 24
8.2 Pranav 25
8.3 Karthik 25
8.4 Bhargav 26
8.5 Nilanjan 26

9 Prompts **27**
9.1 Ankit 27
9.2 Pranav 28
9.3 Karthik 28
9.4 Bhargav 28
9.5 Nilanjan 29

10 Milestones & Timeline **30**

11 Future Scope **30**
11.1 Framework Goals 30
11.2 Prototype Goals 31
11.3 Research Goals 31

I. Problem Statement

Desktop productivity applications accumulate features aggressively over their lifetimes. A modern spreadsheet editor or code-integrated tool may ship with upward of 500 individual commands; telemetry consistently shows that a single user interacts with fewer than 10% of these daily. The remaining 90%—the *bloat*—does not simply sit inert. Its effects are concrete and measurable.

A. The Static UI Architecture Problem

The "one-size-fits-all" static user interface creates distinct, overlapping pain-points for three classes of stakeholders.

Novice and junior users face disproportionate cognitive load. When every feature is always visible, beginners spend 10–15 minutes per day scanning cluttered toolbars simply to locate basic operations. The resulting feature blindness induces a steep and demoralising learning curve that has no functional justification.

Power users suffer from increased task completion time. Specialists with tightly scoped workflows must navigate across multiple tabs dozens of times each session to access the small cluster of tools that comprise their entire working repertoire. HCI research cited in our earlier proposal validates that forcing users to adapt to static software reduces engagement by up to 31% and decreases task efficiency by 22% compared to systems that dynamically match the user's behavior.

System hardware, particularly resource-constrained machines (4 GB RAM laptops, entry-level desktops), bears the most technically severe consequence. Modern UI frameworks load the entire DOM and initialize every backend module at startup regardless of whether the user will ever invoke them. Background sync workers, analytics daemons, and heavy computation threads run indefinitely once the application boots, consuming RAM and CPU for features the user may visit once a month or never.

B. Why Existing Solutions Are Insufficient

Standard workarounds fail in characteristic ways.

- **Manual customization tools** (e.g., Microsoft's Quick Access Toolbar) require explicit user configuration. Industry data shows fewer than 5% of users actively customize their interface; the time and cognitive overhead of customization defeats its own purpose.
- **CSS-based visibility hiding** (`display:none`) removes elements from the rendered view but does not prevent them from being loaded into memory, initialized, or executing background work.
- **Automated runtime debloaters** and static code-deletion tools introduce what we term the *Observer Paradox*.

The Observer Paradox. A background monitoring process that watches application behavior and infers unused code paths—whether a Python AST parser, a real-time WebSocket telemetry pipeline, or a 500 ms polling profiler—can itself consume 150–400 MB of RAM and significant CPU cycles. The cost of observation exceeds the cost of the bloat it is

attempting to eliminate. Furthermore, automated code-deletion tools permanently remove code paths, silently severing dependencies that are not immediately visible to the static analysis tool and causing hard-to-diagnose regressions.

The fundamental problem is not that adaptation is impossible; it is that every prior approach attempts adaptation through a mechanism that contradicts the goal.

II. Proposed Solution

A. Core Philosophy: Starve, Don't Delete

AdaptiveFlow rejects the parallel observer in favor of a two-part design contract:

1. **Developer-enforced architectural discipline** applied at build time, codified as a portable knowledgebase of guidelines.
2. **A flat JSON boolean configuration file** read exactly once at startup, evaluated periodically in the background, and consulted by an in-memory lookup engine during runtime.

The result is zero parallel processing overhead and 100% application stability. No code is deleted from disk. Instead, unused features are *starved* of active compute resources—they are never loaded into RAM, never parsed by the JavaScript engine, never initialized by the class loader. To the operating system and to the user, those features do not exist at runtime, but a developer can restore any of them by flipping a boolean.

B. The Four Core Modules

Development Guidelines (The Foundation)

The framework depends entirely on the application being built to be *intrinsically prunable*. This requires three architectural disciplines.

Modular decoupling: Following the Command Pattern strictly. UI elements contain no business logic; they dispatch commands. Business logic is never entangled with view rendering or database access in the same function.

Lazy loading: Heavy libraries, background threads, and complex engines are never eagerly initialized at startup. They are loaded conditionally, only after the Flipper module confirms the feature is active.

Feature ID tagging: Every independently disableable capability is assigned a globally unique `ID_[DOMAIN]_[CAPABILITY]` string identifier. This identifier is the contract between the UI layer, the tracking system, the usage database, and the configuration schema. A mismatch at any point silently breaks the adaptation cycle for that feature.

Statistics and Tracking Module (The Observer)

A hyper-lightweight tracking mechanism built directly into the application's event loop. It uses passive event delegation on tagged UI elements, creates a minimal two-field payload (`f_id`, `ts`), pushes it to an in-memory queue in O(1), and flushes the queue asynchronously

on a timer or lifecycle event. It adds sub-millisecond overhead per interaction and never performs synchronous I/O on the main thread. No AST parsers, no runtime profilers, no real-time sockets.

The Flipper Module (The Gatekeeper)

The Flipper reads the flat feature-flag JSON schema once at application startup and caches all flags in an in-memory dictionary. At runtime, every feature gate in the codebase queries `isActive(featureId)`, which performs a single dictionary lookup—O(1), synchronous, with no disk or network access. Unknown feature IDs default to `true` to prevent accidental lockout of unregistered features. The Flipper state is immutable during a user's active session, preventing the UI from shifting beneath a user's hands while they are working.

The Weekly Adaptation Cycle (The Evaluator)

A background function—triggered on a cron schedule, on login, or after a fixed number of boots—queries the usage statistics store for the rolling 14-day usage window for each feature. Features that fall below a predefined threshold are flipped to `false` in the JSON schema. Locked safety features (navigation, authentication, settings, the restore UI) are never evaluated and can never be set to `false`. Users may manually restore any hidden feature from the Settings page at any time.

C. The Feature ID Contract

The Feature ID is the single connective tissue linking all four modules. Its format is `ID_SCREAMING_SNAKE_CASE`. Because the four modules are deliberately decoupled and never communicate directly at runtime, there is no mechanism to detect a mismatch. A typo in a Feature ID does not throw a runtime exception—it silently creates a ghost entry in the statistics database that accumulates no usage, while the Flipper treats the mistyped key as `undefined` and defaults to `true`. The feature is never adapted. This is why the Feature Tagging audit is treated as a strict prerequisite: it must be completed and committed to version control before any implementation begins.

D. Deliberate Trade-offs

- **No disk footprint reduction.** All feature code remains on the hard drive. The framework targets active compute (RAM and CPU), not static storage.
- **High engineering discipline required.** This is not a "plug-and-play" script. It requires thoughtful feature tagging, architectural decoupling, and lazy loading at every relevant entry point.
- **No real-time adaptation.** The Flipper state is fixed for the duration of a session. The adaptation cycle is intentionally periodic.
- **Not suitable for legacy monoliths without partial refactoring.** Tightly coupled code that interweaves UI, business logic, and database calls cannot be meaningfully gated without first separating those concerns.

III. The Knowledgebase: An Agent-Optimized Architecture

The primary deliverable of Release 2 is not just working code—it is a *reusable, agent-traversable knowledgebase* that enables any AI coding agent (or human developer) to implement the adaptive framework correctly and efficiently on any target codebase. The knowledgebase is structured to prevent context-window bloat by enforcing a deterministic, sequential file traversal protocol.

A. Directory Structure

```
adaptive_software_kb/
|
|-- 00_Context_and_Goals/
|   |-- 00_goal.md              # Core philosophy & entry point
|   `-- 01_developer_context.md # Context intake & project profiling
|
|-- 01_Architecture/
|   |-- 01_router.md            # Central traffic cop / orchestrator
|   |-- 02_flipper_module.md    # Runtime gating + tracking contract
|   |-- 02_monitor_module.md    # Frontend event capture & batching
|   |-- 03_metafile_schema.md   # Feature flag JSON schema contract
|   |-- 03_statistics_module.md # Backend ingestion & evaluator
|   |-- 04_flipper_logic.md     # Full Flipper implementation spec
|   `-- 05_metadata_schema.md   # Persistence strategies (all stacks)
|
|-- 02_Assessment_and_Tagging/
|   `-- 01_feature_tagging.md   # Feature audit & ID assignment
|
|-- 03_Code_Patterns/
|   |-- 01_frontend_dom.md      # UI component gating (React/Vue/DOM)
|   |-- 02_backend_api.md       # API route gating (Express/FastAPI)
|   |-- 03_background_jobs.md   # Worker/cron gating
|   |-- 04_middleware.md        # Request-level gate middleware
|   |-- 05_asset_manager.md     # Heavy asset preload control
|   |-- 06_db_query_logic.md    # Query fragmentation (SQL/NoSQL)
|   |-- 07_package_imports.md   # Heavy package lazy-import
|   `-- 08_memory_cleanup.md    # Post-removal leak prevention
|
`-- 04_Memory_and_Context/
    |-- 00_memory_protocol.md
    |-- 01_Working_Memory/
    |   |-- 01_active_task_state.md
    |   |-- 02_inter_agent_scratchpad.md
    |   `-- 03_action_log.md
    |-- 02_Orchestrator_Learnings/
    |   `-- 01_routing_heuristics.md
    `-- 03_Subagent_Learnings/
        |-- 01_codebase_quirks.md
        |-- 02_error_ledger.md
        `-- 03_codebase_map.md
```

Listing 1: Knowledgebase directory layout

B. Folder-Level Purposes

Folder	Purpose
`00_Context_and_Goals`	Entry point. Agent reads the core philosophy, then runs the developer context intake to build a Project Context Summary that every downstream agent consumes.
`01_Architecture`	Core framework documentation: the Flipper engine, the Monitor module, the Statistics module, the metadata schema, and the routing orchestrator that directs agents to the correct implementation file.
`02_Assessment_and_Tagging`	Feature audit instructions. Produces the Feature Registry, the initial Metadata Schema, and the implementation Tagging Checklist. Must be completed before any implementation work begins.
`03_Code_Patterns`	Eight implementation templates, one per architectural layer. Each file contains stack-specific code examples, anti-patterns, and an "Observed In" section with real-world application examples drawn from the three prototypes.
`04_Memory_and_Context`	Agent state system following ICM (In-Context Memory) principles. Contains working memory (active task state, inter-agent scratchpad, action log) and long-term learnings (routing heuristics, codebase quirks, error ledger).

Table 1: Knowledgebase folder purposes

C. Agent Traversal Protocol: O(1) Context Scaling

The knowledgebase is explicitly designed to prevent agents from speculatively loading irrelevant files. The enforced traversal protocol is:

1. **Entry through `00_goal.md`.** This file explains the Observer Paradox, the four modules, and the Feature ID contract. It ends with a hard directive: do not read any other file yet. Proceed to the Router.
2. **Context intake via `01_developer_context.md`.** The context agent resolves seven project dimensions (application type, frontend stack, backend stack, database, identity model, performance constraints, and codebase state) and produces a structured Project Context Summary JSON that is attached to all downstream agent calls.
3. **Routing via `01_router.md`.** The Router classifies the developer's request into one of five categories (core infrastructure, assessment, frontend, backend/data, memory/orchestration) and outputs the minimal set of files the agent must read—nothing more.
4. **Targeted implementation.** The agent reads only the routed files, then generates code. It appends outcomes to the action log before handoff.

This architecture guarantees that an agent implementing a frontend UI gate never reads the backend API gating file. An agent auditing a new codebase never reads the memory

cleanup patterns. Token consumption scales with task complexity, not with knowledgebase size.

D. The Router as Orchestrator

`01_router.md` functions as the central nervous system of the knowledgebase. Its routing matrix maps trigger keywords to precise file targets:

Category	Routed To
Core infrastructure (setup, flipper, schema, weekly cycle)	`02_flipper_module.md`, `03_metafile_schema.md`
Assessment and tagging (audit, feature IDs, codebase map)	`02_Assessment_and_Tagging/01_feature_tag`
Frontend (UI, DOM, component, assets, imports, leaks)	`01_frontend_dom.md`, `05_asset_manager.md`, `07_package_imports.md`, `08_memory_cleanup.md`
Backend/data (API, SQL, cron, worker, middleware)	`02_backend_api.md`, `03_background_jobs.md`, `04_middleware.md`, `06_db_query_logic.md`
Orchestration state (handoff, resume, action log)	`04_Memory_and_Context/*`

Table 2: Router classification matrix

E. The Memory and Context System

The `04_Memory_and_Context` folder implements an agent memory layer that prevents repeated work and preserves state across multi-session builds.

Working Memory captures the current active task state, the inter-agent scratchpad for raw handoff payloads, and the action log that records every completed step with its outcome. Before routing any new task, the orchestrator reads the action log to avoid re-running completed steps.

Long-Term Learnings are maintained as two categories of accumulated knowledge: *Orchestrator Learnings* (routing heuristics derived from past sessions) and *Subagent Learnings* (codebase-specific quirks, error ledger entries, and the codebase map). These accumulate over the project lifetime and reduce per-session setup overhead.

F. Technologies and Stack Coverage

The knowledgebase is intentionally stack-agnostic. It explicitly supports the following environments through parameterised implementation templates.

- **Frontend:** React (TypeScript), Vue, Vanilla JS/DOM
- **Backend:** Express.js, FastAPI, Django, Spring Boot
- **Database:** PostgreSQL, MySQL, SQLite, MongoDB, DynamoDB
- **ORM:** Prisma, TypeORM, SQLAlchemy, Sequelize, Mongoose

- **Platform:** Web (SPA and SSR), Desktop (Electron), Mobile (React Native, iOS, Android)
- **Feature gate mechanisms:** JSON boolean config, in-memory singleton, optional Redis-backed distributed flipper, weekly adaptation cron

IV. Prototype Case Studies

Release 2 delivers three distinct prototype implementations, each serving a different validation purpose within the overall research framework.

Prototype	Codebase	Validation Purpose
A	NEXUS (scratch, no KB)	Baseline: a feature-rich application built without adaptive guidelines to establish the pre-adaptation state.
B	NEXUS (scratch, with KB)	Primary: the same application rebuilt with full adaptive integration using the knowledge-base. Demonstrates framework correctness and resource savings.
C	Conduit RealWorld (existing repo, with KB)	Retrofit: the framework applied to an existing, independently authored full-stack application. Demonstrates generalisability.

Table 3: Prototype matrix and validation purposes

A. Prototype A: NEXUS Baseline (Without Knowledgebase)

Application Overview

NEXUS is a premium, full-stack desktop productivity and financial intelligence application. It is intentionally feature-heavy and architecturally layered, with every major subsystem independently operable and clearly modular. It was constructed as the primary testbed for the adaptive framework: feature-rich enough that adaptation produces measurable savings, modular enough that gating can be applied cleanly.

NEXUS bridges the gap between a web application and a native desktop application through a split architecture running entirely locally.

System Architecture

```
+---------------------------------+
| Frontend: React + Vite          |
| Glassmorphic UI, CSS tokens     |
+------------------+--------------+
                   | IPC Bridge
+------------------+--------------+
| Backend: Express (port 4771)    |
| Validation, complex transactions|
| Worker thread orchestration     |
+------------------+--------------+
                   | Prisma ORM
+------------------+--------------+
| Database: SQLite (nexus.db)     |
| Local filesystem storage        |
| Zero cloud dependency           |
+---------------------------------+
```

Listing 2: NEXUS baseline architecture layers

Frontend (React + Vite): A highly responsive glassmorphic UI using custom CSS design tokens defined in `index.css`. The component architecture is modular, with each major feature area contained in a dedicated page component and lazy-loaded from the router.

Backend (Express): An internal API running on a dynamic port (default 4771), responsible for data validation, complex financial transactions, and orchestrating worker threads. The Express layer is the sole interface between the React frontend and the Prisma/SQLite layer.

Database (Prisma + SQLite): A local `nexus.db` file stored in the user's `userData` directory in production. Prisma provides type-safe query access and `$transaction` guarantees for double-entry ledger operations that atomically update account balances when transactions are created.

Background Workers (worker_threads): Heavy computational tasks are completely offloaded from the main Node.js process and the UI render thread, preventing UI freezing during large-scale data operations.

Core Feature Modules

NEXUS is designed as a unified command center with ten interwoven modules.

Module	**Description**
Dashboard	Central hub. Lazy-loads a Recharts financial graph. Displays recent transactions, upcoming tasks, and quick-summary widgets.
Accounts & Transactions	Double-entry style ledger. Creating a transaction atomically adjusts the associated account balance using `$transaction` to prevent race conditions.
Tasks & Time Tracking	Priority-grouped task manager (TODO, IN_PROGRESS, DONE) with active duration logging.
Habits	Consistency tracking grid for building recurring behavioral habits.
Notes & Journal	Sidebar notes with tagging and a daily journal with mood metadata.
Budgets & Goals	Milestone tracking tied to categories or specific financial targets.
Investments	Portfolio tracker that dynamically updates overall Net Worth.
Reports (Worker)	Offloaded to `report.worker.ts`. Aggregates transactions, accounts, and investments across a user-specified date range. Computes cash flow, categorises income/expenses, and calculates real-time Net Worth.
Analytics (Worker & Cron)	Triggered weekly (Sunday midnight) or on demand. Computes three cross-domain performance scores (Financial, Productivity, Habit Consistency) by querying the Transaction, Task, and HabitLog tables. Persists a new AnalyticsData record. The UI fetches the latest computed record.
Feature Usage Tracking	Aggregates FeatureUsageLog telemetry asynchronously through the general worker to maintain a clean database without blocking standard API endpoints.

Table 4: NEXUS module inventory

Heavy Background Work

The Reports Worker (`report.worker.ts`) performs extensive multi-table data aggregation across `Transactions`, `Accounts`, and `Investments`. It fetches thousands of transaction records within a date range, categorises them into INCOME and EXPENSE, and calculates cash flow. It simultaneously aggregates all account balances and computes real-time Net Worth by multiplying investment quantities by their unit prices. By computing this off the main thread, the Electron UI and the Express router remain instantly responsive even when the user has years of financial data.

The Analytics Worker (`analytics.worker.ts`) computes three primary scores:

$$\text{Financial Score} = \frac{\text{Total Income} - \text{Total Expense}}{\text{Total Income}} \times 100$$

$$\text{Productivity Score} = \frac{\text{Completed Tasks}}{\text{Total Tasks}} \times 100$$

$$\text{Habit Consistency} = \frac{\text{Completed Habits}}{\text{Total Habits}} \times 100$$

The general CRON (`cron.ts` and `worker.ts`) handles daily budget threshold evaluations and habit log resets, and aggregates FeatureUsageLog telemetry for the adaptation evaluation pipeline.

Why This Is the Baseline, Not the Goal

Prototype A establishes the pre-adaptation state: all features are always loaded, all workers always start, the analytics cron always runs regardless of whether the user has ever opened the analytics module. On constrained hardware this represents peak resource consumption. It serves as the control against which the adaptive version (Prototype B) is measured.

B. Prototype B: NEXUS Adaptive (With Knowledgebase)

Why Adaptability Matters for NEXUS

No two NEXUS users have the same workflow. A student uses tasks, notes, habits, and time tracking. A freelancer uses accounts, transactions, reports, and time tracking. A household user uses budgets, goals, and journals. A feature-rich application should not force every user profile to carry the same interface and background processing workload. Prototype B applies the full adaptive framework to NEXUS, observing feature usage and adjusting the active runtime feature set at every layer of the architecture.

Feature Registry

The Feature Tagging audit produced the following registry for NEXUS. Feature IDs are divided into locked (always-on safety features) and adaptive (usage-evaluated) categories.

Locked features are always enabled because they are necessary for navigation, safety, and the recovery path:

```
ID_NAV_SIDEBAR
ID_APP_SETTINGS
ID_RESTORE_FEATURES_TAB
ID_ERROR_BOUNDARY
ID_DASHBOARD_VIEW
ID_USAGE_TRACKING
ID_ADAPTATION_WEEKLY_EVALUATOR
```

Listing 3: NEXUS locked feature IDs

Adaptive feature modules are grouped by domain:

Domain	Feature IDs	Threshold
Finance	ID_FINANCE_ACCOUNTS, ID_FINANCE_TRANSACTIONS, ID_FINANCE_BUDGETS, ID_FINANCE_INVESTMENTS	2–3
Productivity	ID_PRODUCTIVITY_TASKS, ID_PRODUCTIVITY_HABITS, ID_PRODUCTIVITY_NOTES, ID_PRODUCTIVITY_GOALS, ID_PRODUCTIVITY_TIME_TRACKING, ID_PRODUCTIVITY_JOURNAL	2–3
Dashboard Widgets	ID_DASHBOARD_FINANCIAL_SUMMARY, ID_DASHBOARD_RECENT_TRANSACTIONS, ID_DASHBOARD_UPCOMING_TASKS, ID_DASHBOARD_GOAL_OVERVIEW, ID_DASHBOARD_JOURNAL_OVERVIEW	1–2
Heavy Frontend	ID_DASHBOARD_ANALYTICS_CHART	1
Reports	ID_REPORTS_GENERATE, ID_REPORTS_EXPORT_JSON	1
Analytics	ID_ANALYTICS_SCORE_VIEW, ID_ANALYTICS_WEEKLY_COMPUTE	1
Background Jobs	ID_SYNC_PERIODIC, ID_BUDGET_DAILY_CHECKS, ID_USAGE_WEEKLY_SUMMARY	0 (manual)

Table 5: NEXUS adaptive feature registry summary

Features with a threshold of 0 are manually controlled only and are not automatically disabled by the weekly evaluator.

Adaptive Metadata Schema

The feature state is stored as a flat JSON schema file (`adaptive_config/feature-flags.local.json`):

```
{
  "schemaVersion": "1.0",
  "userId": "local-profile",
  "lastEvaluated": "1970-01-01T00:00:00.000Z",
  "lockedFeatures": [
    "ID_NAV_SIDEBAR",
    "ID_APP_SETTINGS",
    "ID_RESTORE_FEATURES_TAB",
    "ID_ERROR_BOUNDARY",
    "ID_DASHBOARD_VIEW",
    "ID_USAGE_TRACKING",
    "ID_ADAPTATION_WEEKLY_EVALUATOR"
  ],
  "features": {
    "ID_PRODUCTIVITY_TASKS":              true,
    "ID_PRODUCTIVITY_HABITS":             true,
    "ID_PRODUCTIVITY_NOTES":              true,
    "ID_PRODUCTIVITY_GOALS":              true,
    "ID_PRODUCTIVITY_TIME_TRACKING":      true,
    "ID_PRODUCTIVITY_JOURNAL":            true,
```

```
    "ID_FINANCE_ACCOUNTS":                true,
    "ID_FINANCE_TRANSACTIONS":            true,
    "ID_FINANCE_BUDGETS":                 true,
    "ID_FINANCE_INVESTMENTS":             true,
    "ID_DASHBOARD_FINANCIAL_SUMMARY":     true,
    "ID_DASHBOARD_RECENT_TRANSACTIONS":   true,
    "ID_DASHBOARD_UPCOMING_TASKS":        true,
    "ID_DASHBOARD_GOAL_OVERVIEW":         true,
    "ID_DASHBOARD_JOURNAL_OVERVIEW":      true,
    "ID_DASHBOARD_ANALYTICS_CHART":       true,
    "ID_REPORTS_GENERATE":                true,
    "ID_REPORTS_EXPORT_JSON":             true,
    "ID_ANALYTICS_SCORE_VIEW":            true,
    "ID_ANALYTICS_WEEKLY_COMPUTE":        true,
    "ID_SYNC_PERIODIC":                   true,
    "ID_BUDGET_DAILY_CHECKS":             true,
    "ID_USAGE_WEEKLY_SUMMARY":            true
  }
}
```

Listing 4: NEXUS adaptive metadata schema

Schema invariants: the `features` object is one-level flat for O(1) lookup. `lockedFeatures` entries are always forced `true` by the schema sanitization routine. Missing or unknown IDs default to `true` at runtime (fail-open). Writes are performed via a temporary-file-and-rename strategy to minimize the risk of corruption during power loss.

Adaptation Layers

The adaptive version gates features at every architectural layer.

Layer	What Adapts	Key Files
Feature registry	Feature IDs, categories, thresholds, locked status	`src/adaptive/registry.ts`
Startup bootstrap	Loads schema before app renders	`src/adaptive/bootstrap.ts`
Frontend flipper	O(1) client-side checks, React context	`src/adaptive/flipper.ts`, `AdaptiveProvider.tsx`
Route gating	Full page modules (Tasks, Reports, Analytics)	`src/App.tsx`, `FeatureRoute.tsx`
Sidebar gating	Nav item visibility	`src/components/Layout.tsx`
Widget gating	Dashboard data fetches and UI sections	`src/pages/Dashboard.tsx`
Heavy import gating	Recharts lazy-load	`src/components/HeavyChart.tsx`
Usage tracking	Batched in-	`src/adaptive/featureTracker.ts`

Adaptive Workflow

The lifecycle of adaptation in NEXUS follows a deterministic sequence:

1. Application starts. It reads `adaptive_config/feature-flags.local.json` from disk.
2. Both the client-side Flipper (`src/adaptive/flipper.ts`) and the server-side Flipper (`server/adaptive/flipper.ts`) are initialised with the loaded schema before any routes or UI components mount.
3. Only active routes, sidebar items, and dashboard widgets are rendered. Disabled page modules are never imported; their JavaScript bundles are never parsed by the engine.
4. As the user interacts with tagged features, the client-side tracker batches events (`f_id`, `ts`) into an in-memory queue.
5. The queue flushes to `POST /api/feature-usage/batch` on a timer or on tab-hidden/page-unload lifecycle events.
6. The backend upserts `FeatureUsageLog` rows in SQLite atomically, partitioned by day (time bucket).
7. The weekly adaptation worker queries the rolling 14-day usage window, compares each feature's total against its `adaptationThreshold`, and writes the updated schema back to disk.
8. The next application launch reflects the adapted feature set.
9. At any time, the user may open Settings, see a list of currently hidden features, and restore any of them with a single click.

Flipper Implementation

```cpp
class FlipperEngine {
  private flags: Record<string, boolean> = {};
  private initialized = false;

  init(schema: { features: Record<string, boolean> }): void {
    this.flags = schema.features ?? {};
    this.initialized = true;
  }

  isActive(featureId: string): boolean {
    if (!this.initialized) {
      console.warn("Flipper not initialized. Defaulting TRUE.");
      return true;
    }
    if (this.flags[featureId] === undefined) {
      console.warn(`Unknown FeatureID: ${featureId}. Defaulting TRUE.`);
      return true;
    }
    return this.flags[featureId];
  }

  getAllFlags(): Record<string, boolean> {
    return { ...this.flags };
  }
```

```
  overrideFlag(featureId: string, state: boolean): void {
    this.flags[featureId] = state;
  }
}

export const Flipper = new FlipperEngine();
```

Listing 5: Core Flipper engine (TypeScript)

Weekly Evaluator Pseudocode

```
def run_adaptation_cycle(user_id):
    schema = read_metafile_schema(user_id)
    cutoff = datetime.now() - timedelta(days=14)

    recent_usage = db.query("""
        SELECT feature_id, SUM(usage_count) as total
        FROM feature_usage_stats
        WHERE user_id = ? AND time_bucket >= ?
        GROUP BY feature_id
    """, user_id, cutoff.date())

    for feature_id, current_state in schema["features"].items():
        if feature_id in schema["lockedFeatures"]:
            continue
        threshold = registry.get_threshold(feature_id)
        total = recent_usage.get(feature_id, 0)
        schema["features"][feature_id] = (total >= threshold)

    schema["lastEvaluated"] = datetime.now().isoformat()
    write_metafile_schema_atomic(user_id, schema)
```

Listing 6: Adaptation cycle evaluator

C. Prototype C: Conduit RealWorld App (Retrofit with Knowledgebase)

Application Overview

The Conduit RealWorld application is a publicly available, independently authored full-stack implementation of the Medium-style blogging platform specification. It provides:

- User authentication (register, login, logout, current user profile)
- Article publishing (create, read, update, delete, paginated listings)
- Social features: profiles, follow/unfollow, favorites, comments
- Discovery: tag-based filtering, global feed, personal follow-based feed

Tech stack: React + Vite frontend, Express.js backend, Sequelize ORM, PostgreSQL database. The codebase was sourced from the public GitHub repository `TonyMckes/conduit-realworld-`

This prototype is the critical validation case: the knowledgebase must be robust enough to guide correct adaptive integration into a codebase it was never designed around, written by a developer with no knowledge of the framework.

Retrofitting Strategy

The retrofit followed the knowledgebase traversal protocol strictly, beginning with the Feature Tagging audit before any implementation.

Repository structure after adaptation:

```
conduit-realworld-example-app/
|-- frontend/              # React + Vite UI
|-- backend/               # Express.js + Sequelize
|-- adaptive_config/       # New: governance artifacts
|   |-- feature_registry.json
|   |-- initial_metadata_schema.json
|   `-- tagging_checklist.md
```

Listing 7: Conduit adaptive repository layout

Feature Registry for Conduit

Locked features (always-on, never evaluated):

```
ID_NAV_TOPBAR
ID_AUTH_LOGIN
ID_AUTH_REGISTER
ID_AUTH_LOGOUT
ID_APP_SETTINGS
ID_RESTORE_HIDDEN_FEATURES
ID_ERROR_BOUNDARY
```

Listing 8: Conduit locked feature IDs

Adaptive features (usage-evaluated):

Feature ID	Display Name	Threshold	Category
`ID_DISCOVERY_POPULAR_TAGS`	Popular Tags Sidebar	3	Heavy UI component
`ID_FEED_PERSONAL`	Personal Feed Tab	3	Backend API + Frontend
`ID_PROFILE_FAVORITED_ARTICLES`	Favorited Articles Tab	2	Backend API + Frontend
`ID_ARTICLE_COMMENTS`	Article Comments Section	3	Backend API + Frontend

Table 7: Conduit adaptive feature registry

Per-User Schema Model

Because Conduit is a multi-user web application, the adaptive schema is stored *per user* rather than per machine. The `Users` Sequelize model was extended with an `adaptiveSchema` JSON column (Sequelize's `DataTypes.JSON`).

```
{
  "schemaVersion": "1.0",
  "userId": "<user-uuid>",
```

```
  "lastEvaluated": "1970-01-01T00:00:00.000Z",
  "lockedFeatures": [
    "ID_NAV_TOPBAR", "ID_AUTH_LOGIN",
    "ID_AUTH_REGISTER", "ID_AUTH_LOGOUT",
    "ID_APP_SETTINGS", "ID_RESTORE_HIDDEN_FEATURES",
    "ID_ERROR_BOUNDARY"
  ],
  "features": {
    "ID_DISCOVERY_POPULAR_TAGS":      true,
    "ID_FEED_PERSONAL":               true,
    "ID_PROFILE_FAVORITED_ARTICLES":  true,
    "ID_ARTICLE_COMMENTS":            true
  }
}
```

Listing 9: Per-user Conduit adaptive schema (initial state)

New users are initialised with all features `true`. The `lastEvaluated` timestamp is set to Unix epoch to force first-evaluation eligibility immediately. Schema sanitization (both frontend and backend) catches malformed or missing schemas and returns a safe default.

Usage Tracking Pipeline

The tracker (`frontend/src/adaptive/tracker.js`) listens for `document` click events on elements tagged with `data-feature-id`. Its control rules:

- **Throttle:** maximum one event per feature per 1000 ms (prevents rapid-fire click inflation)
- **Queue cap:** 500 events before forced flush
- **Flush interval:** every 30 seconds via `setInterval`
- **Lifecycle flush:** on tab hidden and before unload using `navigator.sendBeacon` with the `keepalive` flag to guarantee delivery during page close

Events are sent to `POST /api/adaptive/track`. If no authentication header exists, the queue is dropped—guest usage is not persisted, since the schema is per-user.

Database Schema for Usage Statistics

```sql
CREATE TABLE feature_usage_stats (
  user_id     VARCHAR(255)  NOT NULL,
  feature_id  VARCHAR(100)  NOT NULL,
  time_bucket DATE          NOT NULL,  -- e.g. '2026-04-14'
  usage_count INT           DEFAULT 1,
  PRIMARY KEY (user_id, feature_id, time_bucket)
);
```

Listing 10: Conduit feature usage statistics table

The composite primary key enables efficient upsert operations. The ingestion endpoint uses Sequelize's `upsert` (which maps to `INSERT ... ON CONFLICT DO UPDATE`) to atomically increment `usage_count` without read-modify-write race conditions.

Adaptation Cycle Trigger

Because Conduit has no always-running server-side scheduler, the adaptation cycle is triggered asynchronously (`setImmediate`) on two authenticated user flows: user login (`signIn`) and current-user fetch (`currentUser`). The evaluator checks whether the minimum 7-day re-evaluation gap has elapsed before running; if not, it returns immediately without touching the schema.

Dual-Layer Runtime Enforcement

Frontend gating uses the `FeatureGate` component and `useFlipper` hook:

- Popular tags sidebar is hidden when `ID_DISCOVERY_POPULAR_TAGS` is `false`
- Personal feed tab is removed and the feed defaults to global when `ID_FEED_PERSONAL` is `false`
- Favorited articles tab and its requests are disabled when `ID_PROFILE_FAVORITED_ARTICLES` is `false`
- Comments section is hidden when `ID_ARTICLE_COMMENTS` is `false`

Backend gating adds a middleware layer for all gated routes:

```
// Personal feed requires ID_FEED_PERSONAL
router.get('/api/articles/feed',
  authenticate,
  flipperGuard('ID_FEED_PERSONAL'),
  ArticlesController.listFeedArticles
);

// Comments require ID_ARTICLE_COMMENTS
router.get('/api/articles/:slug/comments',
  optionalAuth,
  flipperGuard('ID_ARTICLE_COMMENTS'),
  CommentsController.getComments
);
```

Listing 11: Conduit backend route gating

When a disabled feature is accessed directly (bypassing the UI), the backend returns HTTP 403 with a structured error: `{ "error": "Feature disabled", "featureId": "ID_FEED_PERSONAL" }`. The dual-layer design ensures that even a technically savvy user who manually crafts an API request cannot bypass a feature gate.

Recovery Interface

The `HiddenFeaturesManager` component lists all adaptive features currently set to `false`. Clicking "Restore" calls `overrideFlag` on the frontend Flipper and persists the change via `PUT /api/user` with the updated `adaptiveSchema` embedded in the request body. Locked features are excluded from the restore list—they cannot be disabled by the evaluator and therefore never appear as hidden.

V. System Architecture Comparison

Concern	**Prototype A** (NEXUS Baseline)	**Prototype B** (NEXUS Adaptive)	**Prototype C** (Conduit Retrofit)
Feature flags	None	Flat JSON, local file	JSON column in Users table
Schema scope	N/A	Per machine (local profile)	Per authenticated user
Flipper lookup	N/A	O(1) in-memory singleton	O(1) in-memory per request
Tracking	FeatureUsageLog via cron	Batched tracker, 30s flush	Batched tracker, 30s flush + sendBeacon
Evaluation trigger	N/A	Weekly cron (Sunday 00:00)	Login / currentUser (async, 7-day gap)
Evaluation window	N/A	14 days	14 days
Frontend gate	Always renders	FeatureRoute, FeatureGate, useFlipper	FeatureGate, useFlipper
Backend gate	Always executes	Middleware (flipperGuard)	Middleware (flipperGuard)
Worker control	Always spawns	Gated by Flipper prespawn	N/A (no workers)
Recovery UI	None	Settings page	HiddenFeaturesManager
Locked features	N/A	7 locked IDs	7 locked IDs

Table 8: Side-by-side architecture comparison of all three prototypes

VI. Tech Stack

A. Knowledgebase (Stack-Agnostic Documentation)

The knowledgebase itself is a collection of Markdown files. No runtime dependencies. It is designed to be checked into any project repository and read by AI coding agents or human developers.

B. Prototype B: NEXUS Adaptive (Primary Desktop App)

Layer	Technology	Justification
Desktop container	Electron.js	Wraps the web app as a native offline executable. Provides the IPC bridge for React-to-Node filesystem communication.
Frontend framework	React 18 + TypeScript + Vite	Component model perfectly suited for conditional rendering. Context API used for the global Flipper provider. Vite ensures fast development builds.
Backend runtime	Node.js + Express	Runs the MAPE-K adaptation logic, complex transactions, and worker orchestration outside the main UI render thread.
ORM	Prisma	Type-safe query access with built-in `$transaction` guarantees for double-entry ledger integrity.
Database	SQLite (`nexus.db`)	Local filesystem storage. Zero cloud dependency. Guarantees offline capability and complete data privacy.
Background compute	Node.js `worker_threads`	Offloads report generation and analytics scoring from the UI thread, keeping the interface responsive during heavy computations.
Scheduling	`node-cron`	Weekly adaptation evaluator, daily budget checks, habit log resets.
Adaptive state	Flat JSON file	Written to `userData/adaptive_config/feature-flags.local` via atomic temporary-file-and-rename write strategy.

Table 9: NEXUS Adaptive tech stack

C. Prototype C: Conduit Retrofit (Full-Stack Web App)

Layer	Technology	Notes
Frontend	React + Vite + JavaScript	Pre-existing codebase. Adaptive layer added via tracker, FeatureGate component, useFlipper hook.
Backend	Express.js (Node.js)	Pre-existing. flipperGuard middleware added for gated routes. Adaptive controller added for track/restore endpoints.
ORM	Sequelize	Pre-existing. `adaptiveSchema` JSON column added to User model. `FeatureUsageStat` model added with composite primary key.
Database	PostgreSQL	Upsert via `INSERT ... ON CONFLICT DO UPDATE` for atomic usage count increments.
Adaptive state	JSON column in Users table	Per-user. Read alongside login payload. Sanitized on every read.
Evaluation trigger	Login / currentUser hooks	`setImmediate` async; 7-day minimum gap enforced.

Table 10: Conduit retrofit tech stack

VII. Team Roles

Our team divided Release 2 responsibilities across the knowledgebase design, the two NEXUS prototypes, the Conduit retrofit, and documentation.

Pranav Tej (CS24B057) — Project Lead & Adaptive Architecture

Responsibilities: Overseeing the overall adaptive architecture for the NEXUS prototypes, designing the weekly adaptation cycle, and coordinating the integration of the Flipper module with the Express backend and worker thread system.

Release 2 Contributions: Designed and implemented the NEXUS adaptive cron schedule (`server/cron.ts`), the server-side Flipper middleware (`server/adaptive/middleware.ts`), and the adaptive bootstrap sequence that ensures both client and server flippers are initialized before any routes mount. Resolved integration conflicts between worker thread spawning and Flipper state at the time of dispatch.

Ankit Sinha (CS24B056) — Research Lead & Knowledgebase Architect

Responsibilities: Designing and authoring the full agent-optimized knowledgebase, validating the traversal protocol against both prototype codebases, and producing all Release 2 documentation.

Release 2 Contributions: Authored all files in `00_Context_and_Goals`, `01_Architecture`, `02_Assessment_and_Tagging`, and `04_Memory_and_Context`. Designed the Developer Context Intake protocol and the Project Context Summary JSON schema. Conducted the

Feature Tagging audit for both the NEXUS and Conduit codebases, producing the Feature Registry, Initial Metadata Schema, and Tagging Checklist artifacts. Authored this report.

Karthik (CS24B046) — Frontend Adaptive Integration

Responsibilities: Implementing the frontend adaptive layer for both prototypes: the Flipper context provider, FeatureRoute, FeatureGate, useFlipper hook, and the HiddenFeatureNotice/restore UI.

Release 2 Contributions: Built the React adaptive provider (`src/adaptive/AdaptiveProvider.tsx`), the typed `useFlipper` hook, the `FeatureRoute` gated router wrapper, and the Settings restore interface for both NEXUS and Conduit. Implemented sidebar navigation gating in `Layout.tsx` and widget-level gating in the NEXUS dashboard.

Bhargav (CS24B028) — Statistics Module & Database Layer

Responsibilities: Implementing the usage tracking ingestion pipeline, the time-bucketed database schema, the upsert logic, and the weekly evaluation engine for both prototypes.

Release 2 Contributions: Designed and implemented the `FeatureUsageLog`/`FeatureUsageStat` Prisma and Sequelize models with composite primary keys. Wrote the batched ingestion endpoint (`POST /api/feature-usage/batch` and `POST /api/adaptive/track`), the atomic upsert routines, the 14-day rolling window aggregation queries, and the adaptation evaluation functions in `server/adaptive/adaptationService` for both NEXUS and Conduit.

Nilanjan Singh (CS24B029) — Client Tracker & Electron Integration

Responsibilities: Implementing the client-side Monitor module across both prototypes, and managing the Electron-specific schema persistence and bootstrap sequence in NEXUS.

Release 2 Contributions: Built the frontend tracker (`src/adaptive/featureTracker.ts` and `frontend/src/adaptive/tracker.js`), implementing throttling, queue capping, timed flush, and `sendBeacon` lifecycle flush. Implemented the Electron-side schema reader and atomic-write utility for the local JSON file in NEXUS. Tagged all feature invocation points in both codebases with `data-feature-id` attributes and `useFeatureLog` hook calls as per the Tagging Checklist.

VIII. AI Usage Declaration

A. Ankit

Knowledgebase authoring — AI assistance: approximately 35%

Tool: Claude (Sonnet), ChatGPT

The core philosophy, the Observer Paradox framing, the traversal protocol design, the seven context dimensions, the router classification matrix, the naming conventions, and the anti-pattern lists were authored manually based on the architectural principles validated in Release 1 research. AI assistance was used for: drafting the JSON schema definitions in `03_metafile_schema.md` and `05_metadata_schema.md` (iterating the field constraint list), suggesting edge-case anti-patterns for the Feature Tagging file, and generating the

initial boilerplate for the memory protocol template. All strategic decisions, routing logic, and framework philosophy were human-authored.

Conduit Feature Tagging Audit — AI assistance: 60%

Tool: Claude (Sonnet)

Prompt summary: "Here is the full directory structure and key source files of this Express/React/Sequelize application. Identify all features that are candidates for adaptive hiding. For each, propose a Feature ID following the `ID_[DOMAIN]_[CAPABILITY]` convention, the category, a threshold, and the primary entry points where gates must be inserted. Also identify which features must be permanently locked."

The AI produced the initial feature candidate list. Ankit manually reviewed and corrected granularity (merging over-split IDs, splitting under-specified ones), verified all entry points against the actual codebase, and authored the final Tagging Checklist.

B. Pranav

NEXUS adaptive cron and middleware — AI assistance: 80%

Tool: Gemini

Prompt sequence:

Entry 1 "Here is the current `server/cron.ts` for NEXUS. I need to add a weekly adaptation evaluator that reads the feature-flags JSON, queries the last 14 days of `FeatureUsageLog` records from Prisma, and flips features below threshold to false. Locked features must be skipped. The write must be atomic." Outcome: Useful. Generated the evaluator function and the cron schedule. Needed corrections to the Prisma aggregate query (the AI initially used a suboptimal `findMany` + JS reduce instead of a `groupBy` aggregate).

Entry 2 "The flipperGuard middleware needs to resolve the feature schema from the request context, not from a global singleton, because NEXUS is a single-user local app but we want the architecture to be extensible. Show me how to attach the schema to the Express request in the authentication middleware and read it in flipperGuard." Outcome: Useful. Generated the `req.adaptiveSchema` attachment pattern.

Entry 3 "The worker dispatch in `runWorker.ts` needs to check the Flipper before spawning a worker thread. If the feature is disabled, it should return null instead of spawning. Show the guard." Outcome: Useful with minor edits to the null-return contract.

C. Karthik

Frontend adaptive layer — AI assistance: 85%

Tool: Cursor (Claude-backed)

Entry 1 "Create a React context called `AdaptiveContext` that holds a Flipper instance. Provide a `useFlipper(featureId: string): boolean` hook. The context must be initialized from a schema passed from the Electron main process via `ipcRenderer.invoke`. Default all unknown IDs to true." Outcome: Useful. Generated the provider and hook correctly.

Entry 2 "Now create a `FeatureRoute` component that wraps React Router's `Route`. If the feature ID is inactive, redirect to the dashboard instead of rendering the page. The component must not import the page module unless the feature is active (use `React.lazy` + `Suspense`)." Outcome: Partially used. The initial output used static imports. Corrected to use dynamic `React.lazy` wrapping so the bundle is never loaded.

Entry 3 "Implement the hidden features restore panel in Settings. It must list all features where `isActive` is false and the feature is not locked. Each has a Restore button that calls `overrideFlag` on the Flipper context and persists the change. Style it to match the existing glassmorphic design tokens." Outcome: Useful. Required iteration on the locked-feature filter logic.

D. Bhargav

Statistics module and database layer — AI assistance: 90%

Tool: Gemini

Entry 1 "Generate a Prisma schema addition for a `FeatureUsageLog` model with fields: `id`, `userId`, `featureId`, `timeBucket` (DateTime, day precision), and `usageCount` (Int). The composite unique key is `(userId, featureId, timeBucket)`." Outcome: Useful. Schema generated correctly.

Entry 2 "Write the Express handler for `POST /api/feature-usage/batch`. It receives { `userId, events: [{f_id, ts}]` }. For each event, compute the date bucket from `ts`, then upsert using Prisma's `upsert` with `increment` on `usageCount`. No read-modify-write cycles." Outcome: Useful. The AI correctly used Prisma's `update: { usageCount: { increment: 1 }}` pattern.

Entry 3 "Write the 14-day usage aggregation query using Prisma's `groupBy` on `featureId`, summing `usageCount` where `timeBucket >=` the cutoff date." Outcome: Useful with minor corrections to the `where` clause nesting.

Entry 4 "Repeat entries 1–3 but for Sequelize and PostgreSQL for the Conduit retrofit. Use `Model.upsert` for the ingestion and a raw `sequelize.query` with SUM and GROUP BY for the aggregation." Outcome: Useful.

E. Nilanjan

Client tracker and Electron schema persistence — AI assistance: 80%

Tool: Gemini, Cursor

Entry 1 "Build a TypeScript class `FeatureTracker` with an in-memory event queue. When `track(featureId)` is called, push `{f_id, ts: Date.now()}` to the queue. Throttle: skip if the same `f_id` was pushed within the last 1000 ms. Flush on a 30-second interval. Flush on `visibilitychange` to hidden." Outcome: Useful. The throttle implementation needed adjustment to use a per-feature last-seen timestamp rather than a single global debounce.

Entry 2 "In the Electron main process, implement `readAdaptiveSchema()` that reads the JSON file from `app.getPath('userData')`, parses it, validates the shape (schemaVersion, features, lockedFeatures), and returns a safe default if parsing fails. Implement

`writeAdaptiveSchema(schema)` that writes to a temp file then atomically renames to the real path." Outcome: Useful. Added the IPC handler exposure (`ipcMain.handle`) independently.

Entry 3 "For the Conduit retrofit, generate the `tracker.js` equivalent in plain JavaScript with the global document-click delegate approach. Tag elements via `data-feature-id`. Add `navigator.sendBeacon` as the flush strategy on `beforeunload` and `visibilitychange`." Outcome: Useful.

IX. Prompts

A. Ankit

Prompt 1 **Tool:** Claude (Sonnet)
Prompt: "You are helping me design a knowledgebase for an agent-optimized adaptive software framework. The framework has four modules: Developer Guidelines, a Tracking Monitor, a Flipper (feature flag engine), and a weekly adaptation evaluator. The knowledgebase must be structured so that an AI agent reads *only* the files relevant to its current task and never speculatively scans others. Design the directory hierarchy, explain the purpose of each folder, and describe how the agent traversal protocol should work."
Purpose: Architecture validation for the knowledgebase directory design. Used to stress-test the traversal logic before implementation.
Outcome: Useful. The AI confirmed the numbered-prefix convention and the router-as-orchestrator pattern. Suggested adding the `04_Memory_and_Context` layer for multi-session state, which was not in the original design.

Prompt 2 **Tool:** Claude (Sonnet)
Prompt: "Here is the Conduit RealWorld app directory tree and key source files. Perform a Feature Tagging audit. Identify all features that are candidates for adaptive hiding. For each, propose a Feature ID following the `ID_[DOMAIN]_[CAPABILITY]` naming convention, a category (heavy-ui-component, background-job, heavy-api-computation, heavy-asset, heavy-package-import), an adaptation threshold (integer, minimum uses in 14 days), and the list of file paths where Flipper gates must be added. Separately, identify all features that must be permanently locked."
Purpose: Generating the initial Feature Registry for Conduit.
Outcome: Useful. Produced a solid first draft. Required manual review and corrections to merge over-granular IDs and add the backend `flipperGuard` entry points the AI missed.

Prompt 3 **Tool:** Claude (Sonnet)
Prompt: "Write the `02_Assessment_and_Tagging/01_feature_tagging.md` file for the knowledgebase. It must: explain what a Feature ID is and why it is a contract between four decoupled systems; define the `ID_[DOMAIN]_[CAPABILITY]` naming convention; define the five taggable feature categories with examples; explain what locked features are and list canonical locked examples; define the Feature Registry JSON schema with all required fields; list strict anti-patterns; and specify the three output artifacts the agent must produce. Write it in the style of a system directive for an AI agent."

Purpose: Drafting the feature tagging instruction file.
Outcome: Useful as a structural scaffold. The anti-patterns section and the Single Responsibility Test explanation were manually rewritten for clarity and precision.

B. Pranav

Prompt 1 **Tool:** Gemini
Prompt: "Given this `cron.ts` file for a Node.js/Express backend with Prisma and SQLite, add a weekly adaptation evaluator function. It must: read the feature-flags JSON file at a given path, query the `FeatureUsageStat` table for the sum of `usageCount` per `featureId` where `timeBucket` is within the last 14 days, skip any featureId in the `lockedFeatures` array, flip features below their threshold to false in the schema object, update `lastEvaluated`, and write the result atomically to disk using a temp-file-and-rename pattern."
Purpose: Generating the adaptation evaluator for NEXUS.
Outcome: Useful. Prisma `groupBy` query required manual correction.

Prompt 2 **Tool:** Gemini
Prompt: "Create an Express middleware called `flipperGuard` that takes a `featureId` string parameter. It should read `req.adaptiveSchema` (attached by a preceding auth middleware), call `flipper.isActive(featureId, schema)`, and either call `next()` or return 403 with `{ error: 'Feature disabled', featureId }`."
Purpose: Backend route gating middleware for NEXUS.
Outcome: Useful.

C. Karthik

Prompt 1 **Tool:** Cursor
Prompt: "Create an `AdaptiveProvider.tsx` React context that wraps the app. It should accept a `schema` prop of type `AdaptiveSchema`, build an in-memory dictionary from the `features` field, and expose a `useFlipper(featureId: string): boolean` hook. Unknown IDs must return true. The provider should also expose an `overrideFlag(featureId, state)` method for the restore UI."
Purpose: Global Flipper context for React.
Outcome: Useful.

Prompt 2 **Tool:** Cursor
Prompt: "Create a `FeatureRoute` component that wraps a React Router route. It should accept a `featureId` prop and a `component` prop (lazy-loaded). If `useFlipper(featureId)` is false, redirect to `/dashboard`. The component should not be imported statically; use `React.lazy` and wrap in `Suspense`."
Purpose: Lazy gated routing for NEXUS.
Outcome: Partially used; dynamic import wrapping required correction.

D. Bhargav

Prompt 1 **Tool:** Gemini
Prompt: "Write the Prisma schema for a `FeatureUsageStat` model. Fields: `id` (auto-increment), `userId` (String), `featureId` (String), `timeBucket` (DateTime), `usageCount` (Int default 1). Add a `@@unique` constraint on `[userId, featureId,`

`timeBucket]`."
Outcome: Useful.

Prompt 2 **Tool:** Gemini
Prompt: "Write the Sequelize model for `FeatureUsageStat` for PostgreSQL. The composite primary key is `(userId, featureId, timeBucket)`. Implement an upsert method that, for a given `(userId, featureId, date)`, either inserts with `usageCount = 1` or increments the existing row's `usageCount` by 1 atomically."
Outcome: Useful.

E. Nilanjan

Prompt 1 **Tool:** Gemini
Prompt: "Build a TypeScript class `FeatureTracker` for a React/Electron app. It maintains an in-memory event queue. The `track(featureId)` method pushes {`f_id, ts: Date.now()`} to the queue, but only if the same `f_id` was not tracked within the last 1000 ms (per-feature throttle). The `flush()` method sends the queue to `POST /api/feature-usage/batch` and clears it. Add a 30-second `setInterval` that calls flush, and register flush on `document.addEventListener('visibilitychange')` for when the tab is hidden."
Purpose: Frontend tracker for NEXUS.
Outcome: Useful. Per-feature throttle logic required correction.

Prompt 2 **Tool:** Cursor
Prompt: "In the Electron main process, write two IPC handlers: `ipcMain.handle('adaptive:read-` that reads `feature-flags.local.json` from the user data path, parses it, validates the schema shape, and returns a safe default if the file is missing or corrupt. `ipcMain.handle('adaptive:write-schema', (event, schema))` that serialises and writes to a `.tmp` file then atomically renames to the real path."
Purpose: Electron schema persistence.
Outcome: Useful.

X. Milestones & Timeline

Phase	Milestone & Deliverables	Target Date	Assigned
Release 1	Architecture validation and UI prototypes: MAPE-K engine, three distinct adaptation prototypes over the MS Excel clone testbed. Proposal and Release 1 report.	Mar 17, 2026	All
Sprint 1	Backend infrastructure: core feature-flag JSON parser, Flipper singleton, statistics ingestion endpoint, feature ID tagging checklist for NEXUS.	Apr 01, 2026	Pranav, Ankit, Nilanjan
Release 2	Framework abstraction and full adaptive implementation: knowledgebase complete and committed; Prototype A (NEXUS baseline) complete; Prototype B (NEXUS adaptive) complete with all 12 adaptation layers integrated; Prototype C (Conduit retrofit) complete with dual-layer gating and per-user schema.	Apr 15, 2026	All
Sprint 2	Performance evaluation: RAM/CPU profiling before and after adaptation for both NEXUS and Conduit; user testing for task completion time and cognitive load; quantitative analysis of feature usage patterns across prototype sessions.	Apr 29, 2026	Karthik, Bhargav, Ankit
Final	Code freeze and polish: final bug resolution, knowledgebase review, full `03_Code_Patterns` file set (currently 8 pattern files) verified against both prototypes, production of the project demonstration video, comprehensive documentation.	May 10, 2026	All

Table 11: Project milestones and timeline

XI. Future Scope

A. Framework Goals

- Complete all eight `03_Code_Patterns` implementation files, verified against at least one real prototype each.
- Publish the final `04_Memory_and_Context` working memory system with populated orchestrator learnings and subagent error ledger entries accumulated during the two prototype builds.
- Add a mobile platform chapter to the knowledgebase covering React Native's `AsyncStorage`

schema persistence and iOS/Android lifecycle flush hooks.

B. Prototype Goals

- **Prototype B (NEXUS Adaptive):** Complete production-grade implementation across all 12 adaptation layers listed in the architecture table. Produce quantitative measurements of RAM consumption and startup time with multiple adaptive profiles (all features on vs. a realistic low-usage profile).
- **Prototype C (Conduit):** Complete the full dual-layer implementation. Conduct user testing with simulated divergent usage patterns to validate that the evaluator correctly identifies and hides low-usage features without affecting high-usage ones.
- **Agent Comparison Study:** A formal comparison of code generated by a Codex-class agent *with* and *without* the knowledgebase for the same implementation task. Evaluate: correctness of Feature ID consistency, presence of Observer Paradox violations, adherence to the lazy-loading contract, and correctness of the atomic write pattern.

C. Research Goals

- Quantify and report: the RAM savings achieved by disabling the Recharts heavy chart import (`ID_DASHBOARD_ANALYTICS_CHART`) and the report worker (`ID_REPORTS_GENERATE`) in NEXUS.
- Quantify and report: the reduction in Express route overhead achieved by the `flipperGuard` middleware blocking disabled routes before any database access in Conduit.
- Validate the agent-traversal claim: demonstrate with token-count logs that an agent using the knowledgebase consumes fewer tokens than one without it for an equivalent implementation task.